Kegiatan Belajar 5F.2: *Endpoint Checkout* dan Penguncian Berkas

Hari 17 · 8 JP · UK 1.1, 1.4, dan 1.5 (utama)

Tujuan KB 5F.2. Setelah mengikuti KB ini, Anda mampu membangun endpoint checkout yang memvalidasi ulang harga dan stok di server, menerapkan pola "validasi dulu, baru tulis" dengan penguncian berkas penuh, dan menampilkan halaman konfirmasi.

Kegiatan	**JP**
Mengapa server tidak boleh memercayai klien (recap dan perluasan)	1
Pola "validasi semua dulu, baru tulis semua" pada berkas JSON	2
Praktik: `api/checkout.php`	3
Praktik: mengirim checkout dari frontend dan halaman konfirmasi	1
Cek kemampuan	1
Jumlah	**8**

Materi 1: Mengapa Server Tidak Boleh Memercayai Klien

Data yang dikirim dari browser **bisa diubah siapa saja** sebelum sampai ke server — lewat Console, lewat alat seperti Postman, atau sekadar menulis ulang `fetch()` manual. Bayangkan jika `api/checkout.php` memercayai harga yang dikirim `keranjang.js`:

```
// JANGAN DITIRU — hanya ilustrasi risiko
{ produk_id: 1, harga: 1, jumlah: 10 }   // harga diubah jadi Rp 1 lewat Console
```

Aturannya mutlak: server **wajib** mengambil ulang harga dan stok langsung dari `data/produk.json`, memakai `produk_id` sebagai satu-satunya petunjuk dari klien yang dipercaya — nilai `harga` yang dikirim klien **diabaikan sepenuhnya**. Ini prinsip yang sama persis dengan Bagian B versi basis data — hanya cara mengambil ulang datanya yang berbeda (menelusuri array, bukan `SELECT`).

Materi 2: Pola "Validasi Dulu, Baru Tulis Semua"

Modul 4F sudah membangun `bukaJsonTerkunci()`/`tutupJsonTerkunci()` — kunci dipasang **sebelum** membaca, dilepas **setelah** menulis. Checkout memakai pola ini dengan urutan yang disengaja:

1. **Kunci dan baca** `data/produk.json`.
2. **Validasi SELURUH item** keranjang dulu (produk ada, jumlah valid, stok cukup) — **belum mengubah apa pun**.
3. Jika **ada satu saja** yang gagal, hentikan di sini, lepas kunci **tanpa menulis ulang** — berkas kembali seperti semula, seolah checkout ini tidak pernah terjadi.
4. Jika **semua** lolos validasi, baru kurangi stok di memori, kunci dan tulis `data/transaksi.json` dengan baris pesanan baru, lalu tulis `data/produk.json` dengan stok yang sudah berkurang.

Urutan "validasi semua dulu, baru tulis" inilah yang menggantikan `beginTransaction`/`commit`/`rollBack` pada Bagian B — jika salah satu produk ternyata stoknya kurang, **tidak ada** produk lain dalam pesanan yang sama yang terlanjur dikurangi stoknya.

Praktik 1: Endpoint `api/checkout.php`

```php
<?php
require __DIR__ . "/lib/json_store.php";

header("Content-Type: application/json");

$payload = json_decode(file_get_contents("php://input"), true);

if (!$payload || empty($payload["items"])) {
    http_response_code(400);
    echo json_encode(["error" => "Data pesanan tidak lengkap."]);
    exit;
}

$namaPemesan = trim($payload["nama_pemesan"] ?? "");
$kontak      = trim($payload["kontak"] ?? "");
$items       = $payload["items"];

if ($namaPemesan === "" || $kontak === "") {
    http_response_code(400);
    echo json_encode(["error" => "Nama dan kontak wajib diisi."]);
    exit;
}

$pathProduk    = __DIR__ . "/../data/produk.json";
$pathTransaksi = __DIR__ . "/../data/transaksi.json";

// Kunci produk.json SEPANJANG proses checkout -- dibaca di sini,
// baru ditulis ulang di akhir (atau tidak sama sekali jika validasi gagal).
$dataProduk = bukaJsonTerkunci($pathProduk, $handleProduk);

try {
    $total   = 0;
    $rincian = [];

    foreach ($items as $item) {
        $produkId = (int) $item["produk_id"];
        $jumlah   = (int) $item["jumlah"];

        // Cari produk yang cocok di dalam array -- pengganti "WHERE id = ..."
        $indeks = null;
        foreach ($dataProduk["produk"] as $i => $p) {
            if ($p["id"] === $produkId) { $indeks = $i; break; }
        }

        if ($indeks === null) {
            throw new Exception("Produk tidak ditemukan.");
        }
        if ($jumlah <= 0) {
            throw new Exception("Jumlah pesanan untuk {$dataProduk['produk'][$indeks]['nama']}
tidak valid.");
        }
        if ($dataProduk["produk"][$indeks]["stok"] < $jumlah) {
            throw new Exception("Stok {$dataProduk['produk'][$indeks]['nama']} tidak mencukupi
(tersisa {$dataProduk['produk'][$indeks]['stok']}).");
        }

        // Harga diambil dari BERKAS, bukan dari data yang dikirim klien.
        $hargaAsli = (int) $dataProduk["produk"][$indeks]["harga"];
        $total += $hargaAsli * $jumlah;

        $rincian[] = [
```

```
            "indeks"    => $indeks,
            "produk_id" => $produkId,
            "jumlah"    => $jumlah,
            "harga"     => $hargaAsli,
        ];
    }

    // ===== Seluruh item lolos validasi -- BARU di sini mulai menulis =====

    foreach ($rincian as $r) {
        $dataProduk["produk"][$r["indeks"]]["stok"] -= $r["jumlah"];
    }

    $kodeTransaksi = buatKodeTransaksi();
    $dataTransaksi = bukaJsonTerkunci($pathTransaksi, $handleTransaksi);

    $idBaru = 1;
    foreach ($dataTransaksi as $t) {
        if ($t["id"] >= $idBaru) $idBaru = $t["id"] + 1;
    }

    $dataTransaksi[] = [
        "id"             => $idBaru,
        "kode_transaksi" => $kodeTransaksi,
        "nama_pemesan"   => $namaPemesan,
        "kontak"         => $kontak,
        "total"          => $total,
        "status"         => "baru",
        "dibuat_pada"    => date("Y-m-d H:i:s"),
        "items"          => array_map(function ($r) {
            return [
                "produk_id"       => $r["produk_id"],
                "jumlah"          => $r["jumlah"],
                "harga_saat_beli" => $r["harga"],
            ];
        }, $rincian),
    ];

    tutupJsonTerkunci($handleTransaksi, $dataTransaksi);
    tutupJsonTerkunci($handleProduk, $dataProduk);   // stok yang sudah berkurang ditulis di sini

    echo json_encode([
        "kode_transaksi" => $kodeTransaksi,
        "total"          => $total,
    ]);
} catch (Exception $e) {
    // Validasi gagal SEBELUM tahap penulisan mana pun -- $dataProduk
    // belum pernah diubah, jadi cukup lepas kunci tanpa menulis ulang.
    tutupJsonTerkunci($handleProduk, $dataProduk);
    http_response_code(400);
    echo json_encode(["error" => $e->getMessage()]);
}
```

Telusuri alurnya baris demi baris sebelum lanjut. Perhatikan khususnya: `$dataProduk` hanya diubah (dikurangi stoknya) **setelah** seluruh `foreach` validasi selesai tanpa `throw` — inilah yang membuat blok `catch` selalu aman memanggil `tutupJsonTerkunci($handleProduk, $dataProduk)` dengan data yang **belum berubah sama sekali**, sehingga melepas kunci tanpa pernah menulis ulang isi yang salah.

Praktik 2: Mengirim *Checkout* dari *Frontend*

Perbarui `index.html`, tambahkan formulir di bagian `#keranjang`:

```
<form id="form-checkout">
  <label>Nama Pemesan <input type="text" name="nama_pemesan" required></label>
  <label>Nomor WhatsApp <input type="text" name="kontak" required
placeholder="08xxxxxxxxxx"></label>
  <button type="submit" class="btn">Checkout</button>
</form>
<p id="status-checkout" role="status"></p>
```

Tambahkan di `keranjang.js`:

```
function pasangFormCheckout() {
  const form     = document.getElementById("form-checkout");
  const statusEl = document.getElementById("status-checkout");

  form.addEventListener("submit", async function (e) {
    e.preventDefault();

    const keranjang = ambilKeranjang();
    if (keranjang.length === 0) {
      statusEl.textContent = "Keranjang masih kosong.";
      return;
    }

    const payload = {
      nama_pemesan: form.nama_pemesan.value.trim(),
      kontak: form.kontak.value.trim(),
      items: keranjang.map(function (item) {
        return { produk_id: item.produk_id, jumlah: item.jumlah };
      }),
    };

    statusEl.textContent = "Memproses pesanan...";

    try {
      const respons = await fetch("api/checkout.php", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(payload),
      });
      const hasil = await respons.json();

      if (!respons.ok) {
        statusEl.textContent = hasil.error || "Checkout gagal.";
        return;
      }

      simpanKeranjang([]);
      perbaruiBadgeKeranjang();
      tampilkanKonfirmasi(hasil);
    } catch (err) {
      statusEl.textContent = "Tidak dapat terhubung ke server.";
      console.error(err);
    }
  });
}

function tampilkanKonfirmasi(hasil) {
  const isiEl    = document.getElementById("isi-keranjang");
```

```
  const totalEl  = document.getElementById("total-keranjang");
  const formEl   = document.getElementById("form-checkout");
  const statusEl = document.getElementById("status-checkout");

  formEl.hidden = true;
  totalEl.textContent = "";
  statusEl.textContent = "";

  isiEl.innerHTML = `
    <div class="status" role="status">
      <p><strong>Pesanan berhasil dibuat!</strong></p>
      <p>Kode pesanan Anda: <strong>${esc(hasil.kode_transaksi)}</strong></p>
      <p>Total: <strong>${rupiah(hasil.total)}</strong></p>
      <p>
        <a class="btn" href="api/struk.php?kode=${esc(hasil.kode_transaksi)}" target="_blank">Lihat
Struk (PDF)</a>
        <a class="btn" href="api/struk.php?kode=${esc(hasil.kode_transaksi)}&unduh=1">Unduh Struk
(PDF)</a>
      </p>
    </div>
  `;
}
```

Panggil `pasangFormCheckout();` di dalam `mulai()`.

Uji sekarang: isi keranjang, checkout dengan data valid — pesan sukses dan kode transaksi harus muncul. Buka `data/transaksi.json` **langsung di VS Code** (bukan lewat browser — ingat, folder ini terproteksi `.htaccess` sejak Modul 4F) — baris pesanan baru harus ada. Periksa juga `data/produk.json` — kolom `stok` harus berkurang sesuai jumlah yang dipesan.

Tautan "Lihat/Unduh Struk" belum berfungsi (galat 404 — `api/struk.php` belum ada). Ini disengaja, dibangun di KB 5F.3.

Latihan 2

1. Coba checkout dengan jumlah pesanan **melebihi stok** — pastikan pesan galat jelas muncul, dan **tidak ada** baris baru tersimpan di `transaksi.json` maupun stok yang berubah di `produk.json`.
2. Buka DevTools, tab Network, periksa *request body* permintaan ke `api/checkout.php` — pastikan **tidak ada** kolom `harga` yang terkirim.
3. *Eksperimen keamanan:* sebelum checkout, buka Console, jalankan `simpanKeranjang([{produk_id: 1, slug: "x", nama: "x", harga: 1, jumlah: 2}])`, lalu checkout. Buka `data/transaksi.json`, periksa `total` yang tersimpan — apakah mengikuti harga asli produk `id:1`, atau angka Rp 1 yang disuntikkan lewat Console?
4. *Tantangan:* tambahkan validasi — jika `kontak` tidak berupa angka yang masuk akal, tolak dengan pesan jelas sebelum menyimpan.

Rangkuman KB 5F.2

- Server mengambil ulang harga dan stok dari `data/produk.json` — nilai dari klien hanya dipakai sebagai petunjuk, tidak pernah dipercaya untuk nilai uang.
- Pola "validasi semua dulu, baru tulis semua" menutup celah data rusak separuh jalan, menggantikan `beginTransaction`/`commit`/`rollBack`.
- `$dataProduk` hanya diubah setelah seluruh validasi lolos — inilah yang membuat jalur `catch` selalu aman melepas kunci tanpa menulis ulang apa pun yang keliru.

Cek Kemampuan KB 5F.2

- ☐ Checkout berhasil menulis ke `data/transaksi.json` dan mengurangi stok di `data/produk.json`.
- ☐ Checkout dengan stok tidak cukup ditolak dengan pesan jelas, tanpa mengubah berkas apa pun.
- ☐ Saya membuktikan sendiri (Latihan 2 butir 3) bahwa harga yang dimanipulasi di `localStorage` tidak memengaruhi `total` yang tersimpan.
- ☐ Saya dapat menjelaskan urutan "validasi dulu, baru tulis" dengan kata sendiri.

Kesalahan umum: mengurangi stok **sebelum** validasi item berikutnya selesai (urutan harus: validasi semua dulu, baru tulis semua); lupa `tutupJsonTerkunci()` di jalur `catch` (kunci tertinggal, checkout berikutnya macet menunggu); membaca `$_POST` alih-alih `php://input`.